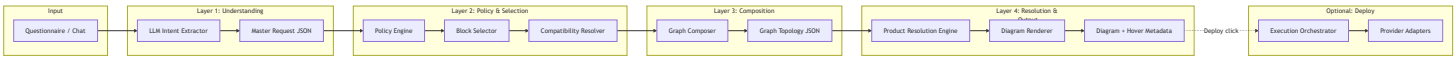
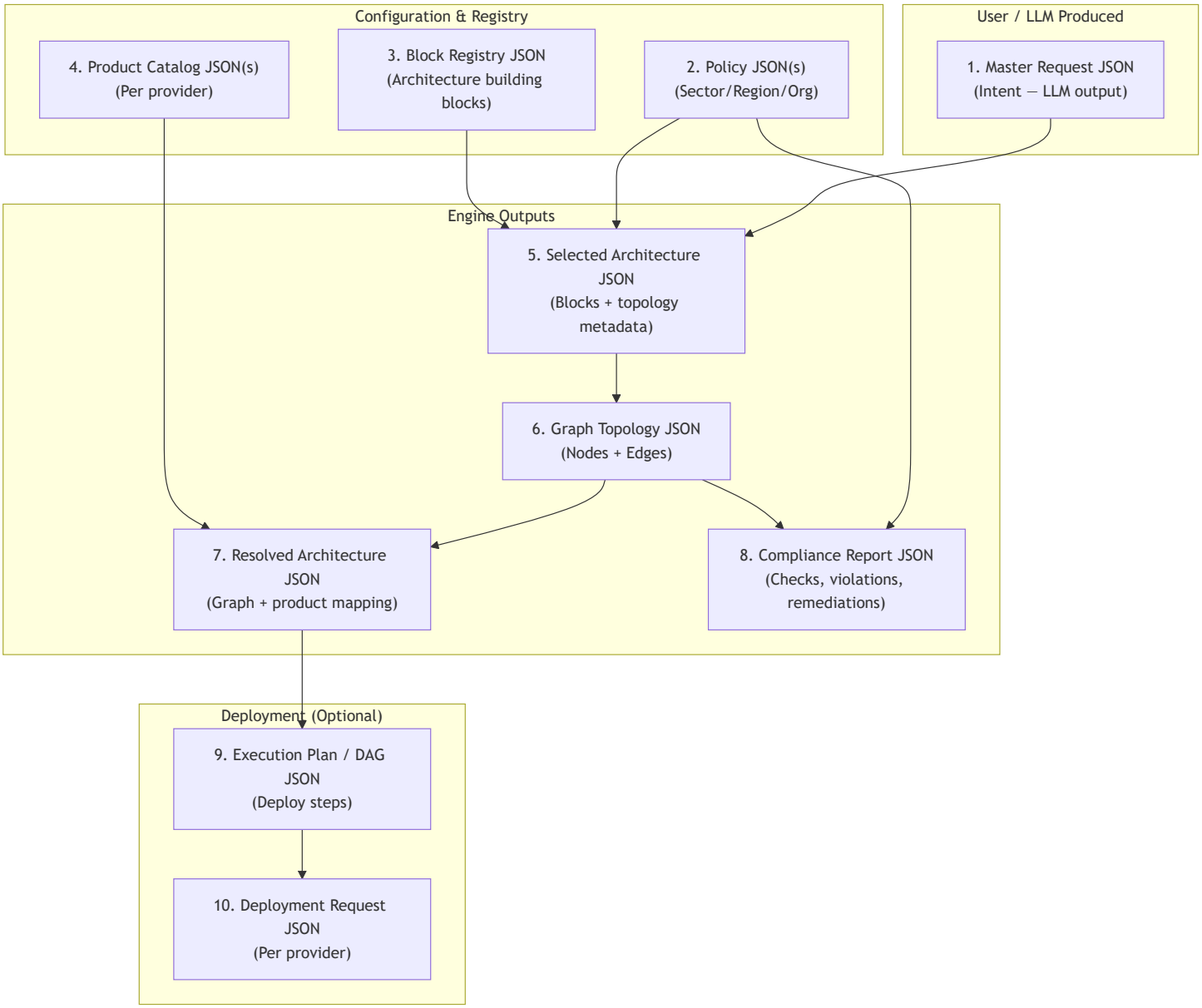
Architecture Synthesis Platform — Complete Plan & JSON Schema Map

This document describes the full plan for the **hybrid, policy-aware, graph-based architecture synthesis** system, including all JSON types and their relationships, in Mermaid form.

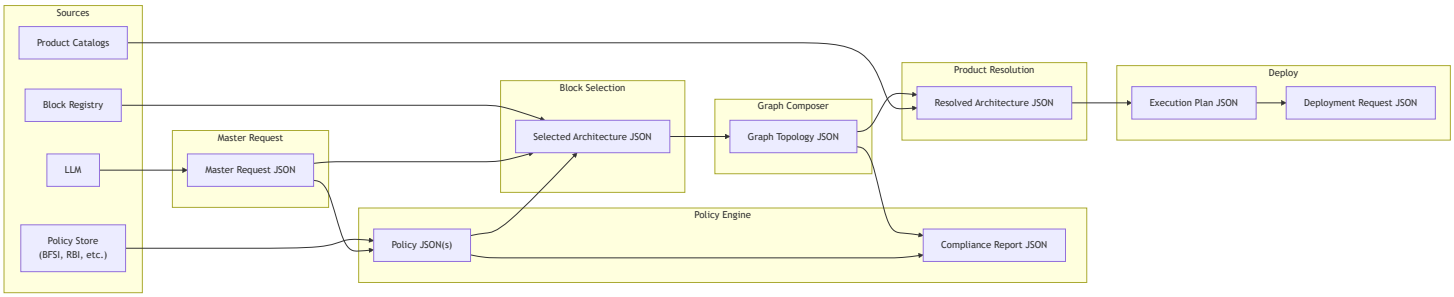
1. High-Level Pipeline Flow



2. All JSON Types — Overview

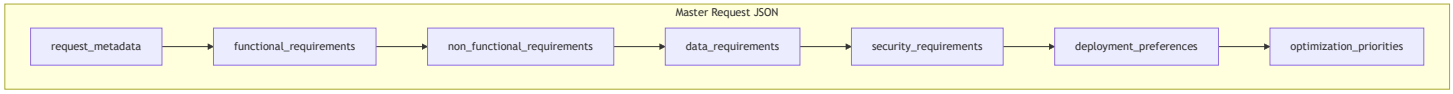


3. Detailed Data Flow (Where Each JSON Is Used)



4. JSON #1 — Master Request JSON (LLM Output)

Purpose: Structured intent from questionnaire/chat. LLM fills this; rest of pipeline is deterministic.



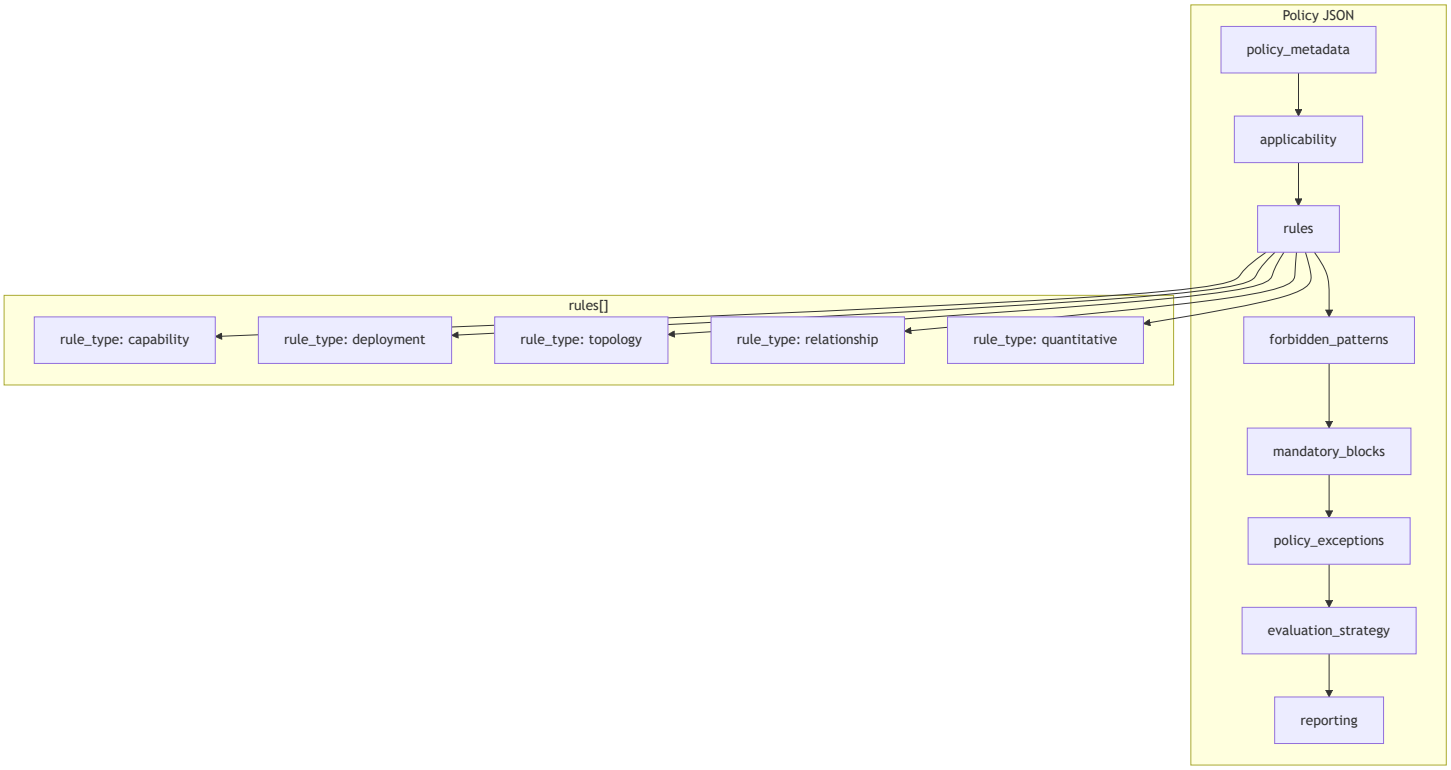
Used by: Policy Engine, Block Selector, Graph Composer.

Schema outline:

- `request_metadata` : request_id, timestamp, environment, sector, business_criticality
- `functional_requirements` : application_type, interaction_model, real_time_processing, third_party_integrations
- `non_functional_requirements` : availability_target_percent, latency_p95_ms, expected_rps_peak, multi_region_required, disaster_recovery
- `data_requirements` : data_types, consistency_model, retention_years, analytics_required
- `security_requirements` : authentication_required, authorization_model, encryption, audit_logging
- `deployment_preferences` : cloud_provider_preference, containerization_preferred, infrastructure_as_code_required
- `optimization_priorities` : performance_weight, cost_weight, compliance_weight, operational_simplicity_weight

5. JSON #2 — Policy JSON (Unified Schema, Sector-Wise Files)

Purpose: Machine-readable compliance rules. One schema; multiple files (BFSI, healthcare, RBI, org).



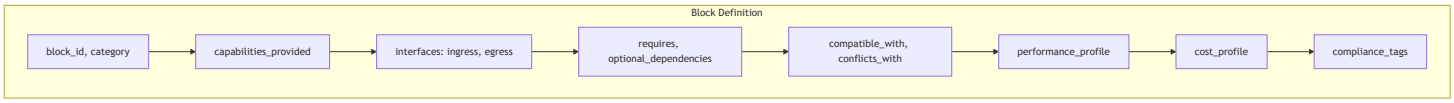
Rule types:

- **capability:** node must support X (e.g. encryption_at_rest)
- **deployment:** region_count ≥ 2, replication_required, data_residency
- **topology:** no public DB, layer placement
- **relationship:** WAF must precede API gateway
- **quantitative:** latency_p95 ≤ 300, cost limits

File layout: `/policies/sectors/bfsi.json`, `banking.json`, `healthcare.json`; `/policies/regional/rbi_india.json`; same schema throughout.

6. JSON #3 — Block Registry JSON (Architecture Building Blocks)

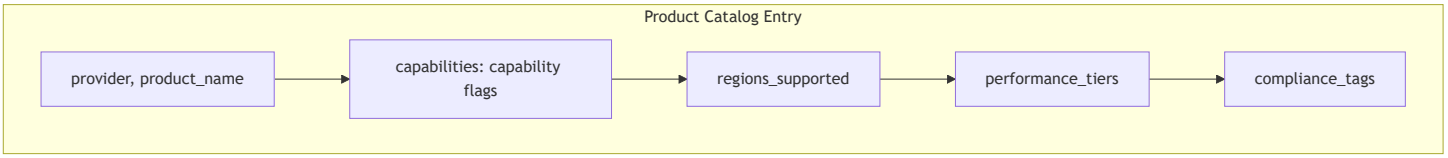
Purpose: Define composable blocks with capabilities, interfaces, and compatibility.



Used by: Block Selector, Compatibility Resolver, Graph Composer (for interfaces/connections).

7. JSON #4 — Product Catalog JSON (Per Provider)

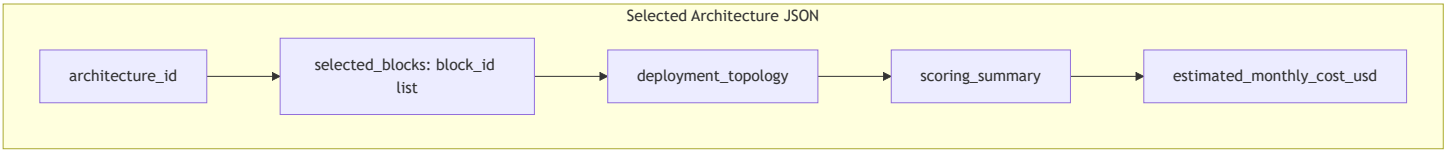
Purpose: Map capability → actual product names (your cloud, AWS, Azure, etc.) for diagram hover and deploy.



Used by: Product Resolution Engine. One catalog (or file) per provider.

8. JSON #5 — Selected Architecture JSON (After Block Selection + Compliance)

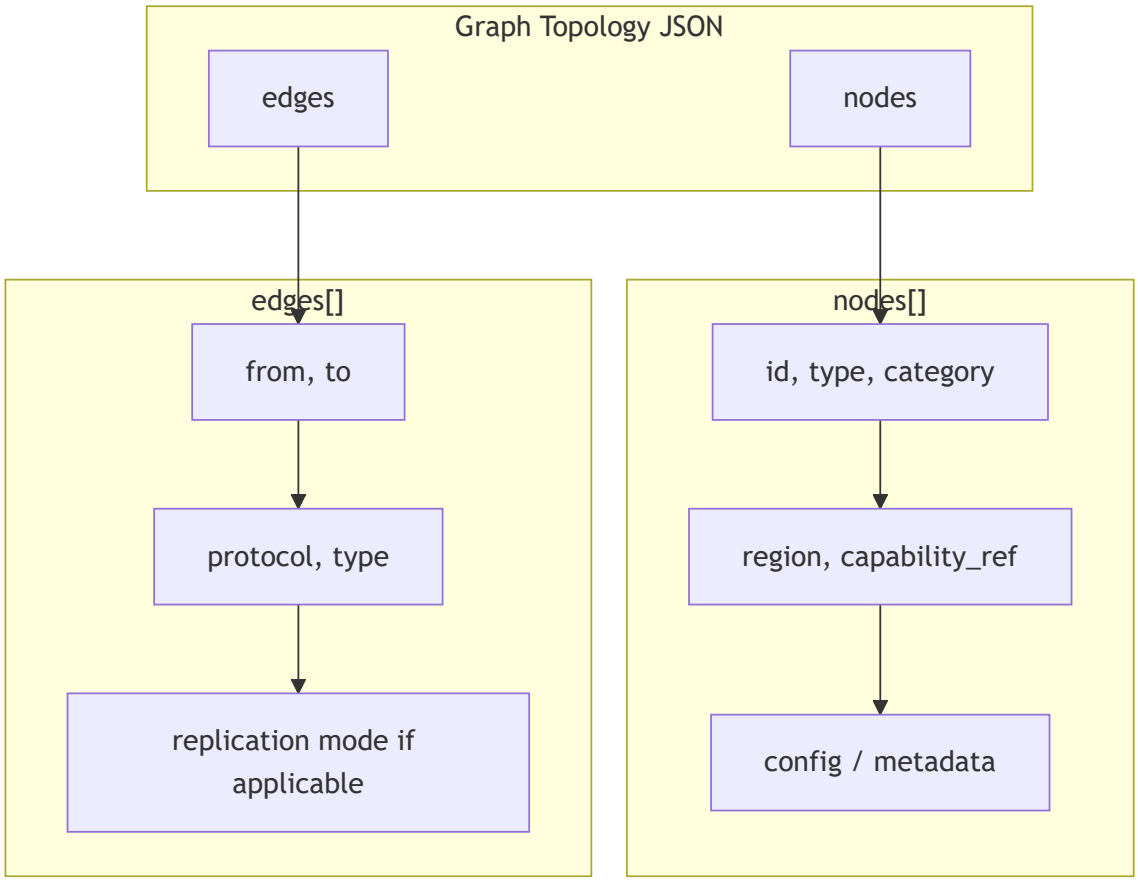
Purpose: List of chosen blocks + deployment topology + scoring summary before graph is built.



Used by: Graph Composer, Compliance Report (if re-run after remediation).

9. JSON #6 — Graph Topology JSON (Composer Output)

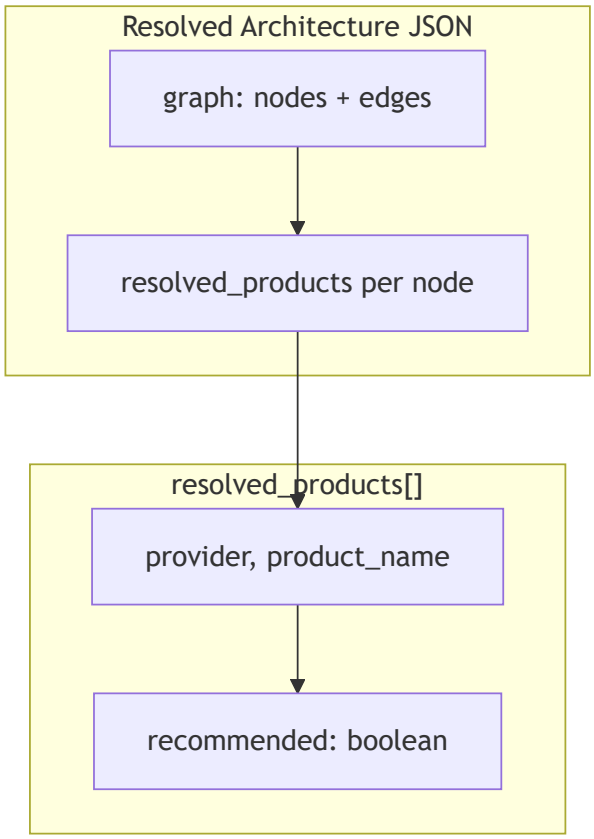
Purpose: Actual architecture as a graph (nodes + edges) for diagram and downstream use.



Used by: Diagram Renderer, Product Resolution (per-node), Compliance (topology/deployment rules), Execution Planner.

10. JSON #7 — Resolved Architecture JSON (Graph + Products)

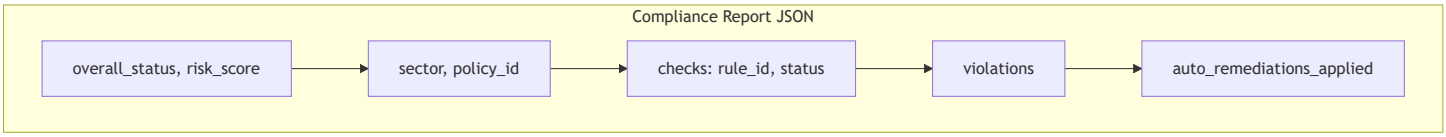
Purpose: Graph plus per-node resolved products (your cloud + alternatives) for diagram and deploy.



Used by: Diagram Renderer (hover), Deployment Orchestrator.

11. JSON #8 — Compliance Report JSON

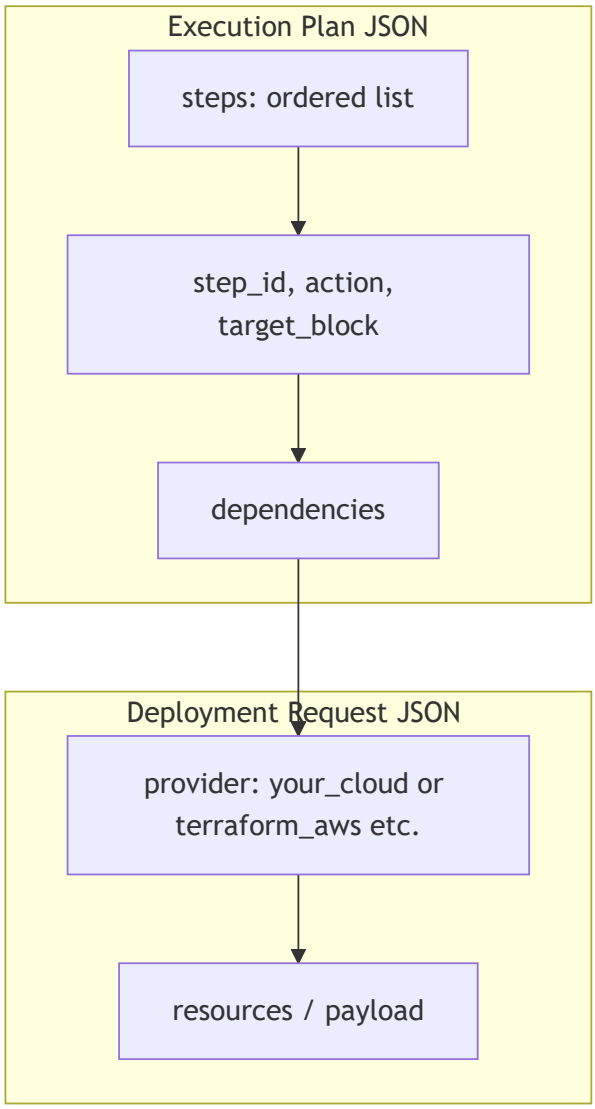
Purpose: Result of policy evaluation: status, checks, violations, auto-remediations.



Used by: UI, audit, and optionally re-input to Policy Engine for re-evaluation after remediation.

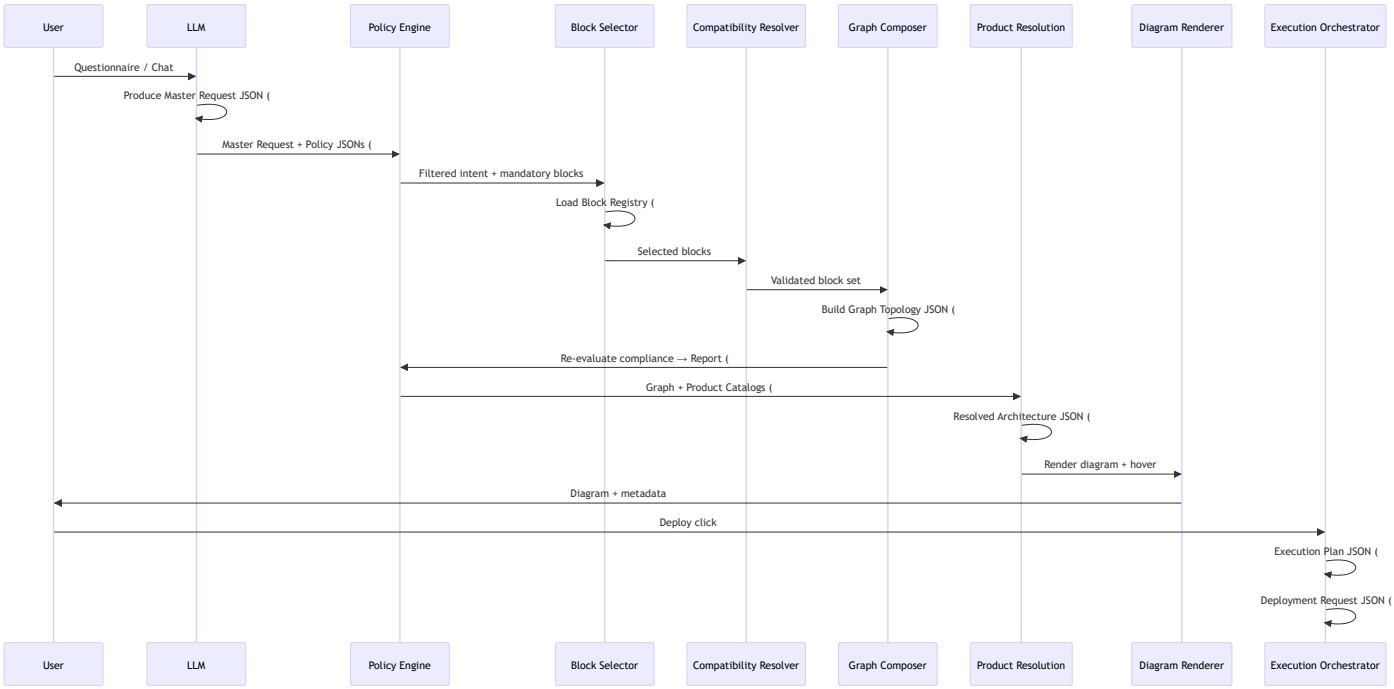
12. JSON #9 & #10 — Execution Plan & Deployment Request (Deploy Layer)

Purpose: Order of provisioning (DAG) and provider-specific payloads.



Used by: Execution Orchestrator, Provider Adapters (your APIs vs Terraform).

13. End-to-End Sequence (All JSONs in Order)



14. Summary Table — All JSONs

#	JSON Name	Producer	Consumer	Purpose
1	Master Request	LLM	Policy Engine, Block Selector	Structured intent
2	Policy	Config (sector/region/org files)	Policy Engine, Compliance	Rules (capability, deployment, topology, relationship, quantitative)
3	Block Registry	Config	Block Selector, Compatibility, Graph Composer	Building blocks with interfaces
4	Product Catalog	Config (per provider)	Product Resolution Engine	Capability → product name
5	Selected Architecture	Block Selector + Compliance	Graph Composer	Chosen blocks + topology metadata
6	Graph Topology	Graph Composer	Diagram, Compliance, Product Resolution, Deploy	Nodes + edges

#	JSON Name	Producer	Consumer	Purpose
7	Resolved Architecture	Product Resolution Engine	Diagram Renderer, Deploy	Graph + product mapping
8	Compliance Report	Policy Engine	UI, Audit	Status, violations, remediations
9	Execution Plan	Execution Orchestrator	Provider Adapters	Deploy DAG
10	Deployment Request	Execution Orchestrator	Your APIs / Terraform	Per-provider payload

15. Complete JSON Examples (All Scenarios)

Below are full, scenario-covering JSON examples for every type used in the pipeline.

15.1 Master Request JSON (LLM Output — All Scenarios)


```json
{
  "request_metadata": {
    "request_id": "req_001",
    "timestamp": "2026-02-26T10:00:00Z",
    "environment": "production",
    "sector": "BFSI",
    "business_criticality": "high"
  },
  "functional_requirements": {
    "application_type": "web_application",
    "architecture_style_preference": null,
    "interaction_model": "synchronous",
    "real_time_processing": true,
    "batch_processing": false,
    "public_access_required": true,
    "api_required": true,
    "third_party_integrations": ["payment_gateway", "email_service"]
  },
  "non_functional_requirements": {
    "availability_target_percent": 99.95,
    "latency_p95_ms": 200,
    "expected_rps_peak": 2500,
    "daily_active_users": 500000,
    "horizontal_scaling_required": true,
    "multi_region_required": true,
    "disaster_recovery": {
      "rto_minutes": 30,
      "rpo_minutes": 5
    }
  },
  "data_requirements": {
    "data_types": ["structured", "unstructured"],
    "initial_volume_gb": 500,
    "monthly_growth_gb": 50,
    "consistency_model": "eventual",
    "retention_years": 7,
    "analytics_required": true,
    "real_time_analytics": false
  },
  "security_requirements": {
    "authentication_required": true,
    "authorization_model": "rbac",
    "data_encryption_at_rest": true,
    "data_encryption_in_transit": true,
    "audit_logging_required": true
  },
```

```
  "deployment_preferences": {
    "cloud_provider_preference": "aws",
    "multi_region_required": true,
    "on_prem_required": false,
    "hybrid_cloud": false,
    "containerization_preferred": true,
    "serverless_preferred": false,
    "infrastructure_as_code_required": true
  },
  "observability_requirements": {
    "logging_required": true,
    "distributed_tracing": true,
    "metrics_monitoring": true,
    "alerting_required": true
  },
  "optimization_priorities": {
    "performance_weight": 0.4,
    "cost_weight": 0.2,
    "compliance_weight": 0.3,
    "operational_simplicity_weight": 0.1
  },
  "budget_constraints": {
    "monthly_budget_usd": 20000,
    "cost_optimization_priority": "medium"
  },
  "future_growth_projection": {
    "expected_user_growth_percentage_per_year": 50,
    "global_expansion_expected": true
  }
}
```

15.2 Policy JSON (Unified Schema — BFSI Example, All Rule Types)


```
{
  "policy_metadata": {
    "policy_id": "bfsi_baseline_v1",
    "policy_name": "BFSI Sector Baseline Policy",
    "sector": "BFSI",
    "region": "India",
    "version": "1.0.0",
    "status": "active",
    "priority": 1,
    "enforcement_mode": "strict",
    "created_at": "2026-02-26T00:00:00Z",
    "updated_at": "2026-02-26T00:00:00Z",
    "description": "Baseline compliance policy for BFSI workloads"
  },
  "applicability": {
    "environments": ["production"],
    "business_criticality": ["medium", "high"],
    "cloud_providers": ["aws", "azure", "gcp"]
  },
  "rules": [
    {
      "rule_id": "data_encryption_at_rest_required",
      "description": "All data storage must support encryption at rest",
      "rule_type": "capability",
      "target_category": "data",
      "severity": "critical",
      "condition": {
        "field": "data_encryption_at_rest",
        "operator": "equals",
        "value": true
      },
      "auto_remediation": {
        "enabled": true,
        "action": "add_or_replace_block",
        "parameters": {
          "required_capability": "data_encryption_at_rest"
        }
      }
    },
    {
      "rule_id": "data_encryption_in_transit_required",
      "description": "All data in transit must be encrypted",
      "rule_type": "capability",
      "target_category": "data",
      "severity": "critical",
      "condition": {
```

```
    "field": "data_encryption_in_transit",
    "operator": "equals",
    "value": true
  }
},
{
  "rule_id": "data_multi_region_required",
  "description": "Data must be deployed in at least 2 distinct regions",
  "rule_type": "deployment",
  "target_category": "data",
  "severity": "critical",
  "condition": {
    "field": "region_count",
    "operator": "greater_or_equal",
    "value": 2
  },
  "additional_constraints": {
    "replication_required": true,
    "replication_mode": "active_active",
    "failure_tolerance": "single_region_failure"
  },
  "auto_remediation": {
    "enabled": true,
    "action": "add_replication_region",
    "parameters": {
      "min_regions": 2
    }
  }
},
{
  "rule_id": "no_public_database_access",
  "description": "Databases must not be publicly accessible",
  "rule_type": "topology",
  "target_category": "data",
  "severity": "critical",
  "condition": {
    "field": "public_access",
    "operator": "equals",
    "value": false
  }
},
{
  "rule_id": "waf_required_for_public_api",
  "description": "Public APIs must be protected by WAF",
  "rule_type": "relationship",
  "target_category": "api_gateway",
  "severity": "high",
  "condition": {
    "must_be_preceded_by": "waf_layer"
```

```
    },
    "auto_remediation": {
      "enabled": true,
      "action": "insert_block_before",
      "parameters": {
        "block": "waf_layer"
      }
    }
  },
  {
    "rule_id": "latency_threshold",
    "description": "P95 latency must be under 300ms",
    "rule_type": "quantitative",
    "target_category": "compute",
    "severity": "medium",
    "condition": {
      "field": "latency_p95_ms",
      "operator": "less_or_equal",
      "value": 300
    }
  },
  {
    "rule_id": "data_residency_constraint",
    "description": "Data must reside within allowed regions",
    "rule_type": "deployment",
    "target_category": "data",
    "severity": "critical",
    "condition": {
      "field": "region",
      "operator": "in",
      "value": ["ap-south-1", "ap-southeast-1"]
    }
  },
  {
    "rule_id": "audit_logging_required",
    "description": "Audit logging must be enabled",
    "rule_type": "capability",
    "target_category": "security",
    "severity": "high",
    "condition": {
      "field": "audit_logging",
      "operator": "equals",
      "value": true
    }
  }
],
"forbidden_patterns": [
  {
    "pattern_id": "single_point_of_failure_db",
```

```
      "description": "Single-region database deployment is not allowed",
      "detection_logic": {
        "category": "data",
        "region_count": 1
      },
      "severity": "critical"
    },
    {
      "pattern_id": "public_database",
      "description": "Database with public access is forbidden",
      "detection_logic": {
        "category": "data",
        "public_access": true
      },
      "severity": "critical"
    }
  ],
  "mandatory_blocks": [
    {
      "block_id": "centralized_logging",
      "reason": "Audit logging required for BFSI"
    },
    {
      "block_id": "key_management_service",
      "reason": "Encryption key control required"
    }
  ],
  "policy_exceptions": [
    {
      "exception_id": "non_production_waiver",
      "applies_to": ["non_production"],
      "expires_at": "2026-12-31T00:00:00Z"
    }
  ],
  "evaluation_strategy": {
    "conflict_resolution": "highest_priority_wins",
    "rule_aggregation": "all_must_pass",
    "custom_plugins_allowed": true
  },
  "reporting": {
    "generate_violation_report": true,
    "include_auto_remediation_details": true,
    "include_risk_score": true
  }
}
```

15.3 Block Registry JSON (Example Blocks — Compute, Data,

Messaging, Security)


```
{
  "blocks": [
    {
      "block_id": "microservices_compute",
      "category": "compute",
      "capabilities_provided": {
        "stateless_compute": true,
        "horizontal_scaling": true,
        "container_support": true
      },
      "interfaces": {
        "ingress": ["http", "grpc"],
        "egress": ["db_connection", "event_publish", "cache"]
      },
      "requires": ["network_layer"],
      "optional_dependencies": ["load_balancer"],
      "compatible_with": ["event_streaming_layer", "nosql_database", "distributed_cache"],
      "conflicts_with": ["monolithic_single_instance"],
      "performance_profile": {
        "max_rps_supported": 10000,
        "latency_p95_ms": 50
      },
      "cost_profile": {
        "base_monthly_cost_usd": 2000,
        "cost_per_million_requests_usd": 5
      },
      "compliance_tags": ["BFSI", "PCI-DSS"],
      "complexity_score": 6
    },
    {
      "block_id": "managed_nosql_db",
      "category": "data",
      "capabilities_provided": {
        "structured_storage": true,
        "eventual_consistency": true,
        "horizontal_scaling": true,
        "data_encryption_at_rest": true,
        "data_encryption_in_transit": true,
        "audit_logging": true,
        "multi_az_deployment": true
      },
      "interfaces": {
        "ingress": ["tcp"],
        "egress": []
      },
      "requires": ["private_network"],
```

```
    "conflicts_with": ["public_database_access"],
    "performance_profile": {
      "max_rps_supported": 100000,
      "latency_p95_ms": 10
    },
    "cost_profile": {
      "base_monthly_cost_usd": 1500,
      "cost_per_million_requests_usd": 2
    },
    "compliance_tags": ["BFSI", "PCI-DSS"],
    "complexity_score": 5
  },
  {
    "block_id": "event_streaming_layer",
    "category": "messaging",
    "capabilities_provided": {
      "asynchronous_processing": true,
      "event_driven_architecture": true,
      "high_throughput_streaming": true
    },
    "interfaces": {
      "ingress": ["tcp", "producer_api"],
      "egress": ["consumer_api"]
    },
    "requires": ["compute_layer"],
    "optional_dependencies": ["schema_registry"],
    "compatible_with": ["microservices_compute", "nosql_database", "distributed_cache"],
    "conflicts_with": ["monolithic_single_instance"],
    "performance_profile": {
      "max_throughput_per_sec": 50000,
      "latency_p95_ms": 20
    },
    "cost_profile": {
      "base_monthly_cost_usd": 2000,
      "cost_per_million_events_usd": 5
    },
    "compliance_tags": ["BFSI"],
    "complexity_score": 8
  },
  {
    "block_id": "waf_layer",
    "category": "security",
    "capabilities_provided": {
      "web_application_firewall": true,
      "ddos_protection": true,
      "audit_logging": true
    },
    "interfaces": {
      "ingress": ["https"],
```

```
        "egress": ["https"]
      },
      "requires": [],
      "compatible_with": ["api_gateway", "load_balancer"],
      "conflicts_with": [],
      "performance_profile": {
        "max_rps_supported": 50000,
        "latency_p95_ms": 5
      },
      "cost_profile": {
        "base_monthly_cost_usd": 500
      },
      "compliance_tags": ["BFSI", "PCI-DSS"],
      "complexity_score": 3
    },
    {
      "block_id": "api_gateway",
      "category": "network",
      "capabilities_provided": {
        "api_management": true,
        "rate_limiting": true,
        "routing": true
      },
      "interfaces": {
        "ingress": ["https", "http"],
        "egress": ["http", "grpc"]
      },
      "requires": ["waf_layer"],
      "compatible_with": ["microservices_compute", "load_balancer"],
      "conflicts_with": []
    }
  ]
}
```


15.4 Product Catalog JSON (Per Provider — Your Cloud, AWS, Azure)

Your cloud (e.g. YC):

```
{
  "provider": "your_cloud",
  "provider_display_name": "Your Cloud",
  "products": [
    {
      "product_id": "yc-vm-standard",
      "product_name": "YC Standard VM",
      "category": "compute",
      "capabilities": {
        "stateless_compute": true,
        "horizontal_scaling": true,
        "container_support": true
      },
      "regions_supported": ["ap-south-1", "ap-southeast-1"],
      "performance_tiers": ["standard", "high"],
      "compliance_tags": ["BFSI"],
      "monthly_cost_usd_estimate": 200
    },
    {
      "product_id": "yc-managed-nosql",
      "product_name": "YC Managed NoSQL",
      "category": "data",
      "capabilities": {
        "structured_storage": true,
        "eventual_consistency": true,
        "data_encryption_at_rest": true,
        "data_encryption_in_transit": true,
        "audit_logging": true,
        "multi_az_deployment": true
      },
      "regions_supported": ["ap-south-1", "ap-southeast-1"],
      "compliance_tags": ["BFSI", "PCI-DSS"],
      "monthly_cost_usd_estimate": 1500
    }
  ]
}
```


AWS:

```
{
  "provider": "aws",
  "provider_display_name": "Amazon Web Services",
  "products": [
    {
      "product_id": "aws-ec2",
      "product_name": "Amazon EC2",
      "category": "compute",
      "capabilities": {
        "stateless_compute": true,
        "horizontal_scaling": true,
        "container_support": true
      },
      "regions_supported": ["ap-south-1", "us-east-1", "us-west-2"],
      "performance_tiers": ["standard", "high"],
      "compliance_tags": ["BFSI", "PCI-DSS"],
      "monthly_cost_usd_estimate": 250
    },
    {
      "product_id": "aws-dynamodb",
      "product_name": "Amazon DynamoDB",
      "category": "data",
      "capabilities": {
        "structured_storage": true,
        "eventual_consistency": true,
        "data_encryption_at_rest": true,
        "data_encryption_in_transit": true,
        "audit_logging": true,
        "multi_az_deployment": true
      },
      "regions_supported": ["ap-south-1", "us-east-1"],
      "compliance_tags": ["BFSI", "PCI-DSS"],
      "monthly_cost_usd_estimate": 1200
    }
  ]
}
```

15.5 Selected Architecture JSON (After Block Selection + Compliance)


```json
{
  "architecture_id": "arch_001",
  "selected_blocks": [
    "waf_layer",
    "api_gateway",
    "microservices_compute",
    "event_streaming_layer",
    "managed_nosql_db",
    "distributed_cache",
    "object_storage",
    "centralized_logging",
    "key_management_service",
    "multi_region_deployment"
  ],
  "deployment_topology": {
    "regions": ["ap-south-1", "ap-southeast-1"],
    "active_active": true,
    "multi_az": true
  },
  "scoring_summary": {
    "performance_score": 0.92,
    "cost_score": 0.75,
    "compliance_score": 1.0,
    "simplicity_score": 0.65,
    "final_weighted_score": 0.87
  },
  "estimated_monthly_cost_usd": 18500,
  "compliance_alignment": ["BFSI", "PCI-DSS"]
}
```

15.6 Graph Topology JSON (Composer Output — Nodes + Edges)


```
{
  "graph": {
    "nodes": [
      {
        "id": "client",
        "type": "external",
        "category": "external",
        "data": {
          "label": "Client",
          "description": "End user / browser"
        }
      },
      {
        "id": "waf_1",
        "type": "security",
        "category": "security",
        "region": "ap-south-1",
        "capability_ref": "waf_layer",
        "data": {
          "label": "WAF",
          "description": "Web Application Firewall"
        }
      },
      {
        "id": "api_gw_1",
        "type": "network",
        "category": "network",
        "region": "ap-south-1",
        "capability_ref": "api_gateway",
        "data": {
          "label": "API Gateway",
          "description": "API management and routing"
        }
      },
      {
        "id": "compute_1",
        "type": "compute",
        "category": "compute",
        "region": "ap-south-1",
        "capability_ref": "microservices_compute",
        "data": {
          "label": "Application Servers",
          "description": "Microservices cluster"
        },
        "config": {
          "scale_type": "horizontal",
```

```
      "min_instances": 2,
      "max_instances": 10
    }
  },
  {
    "id": "event_stream_1",
    "type": "messaging",
    "category": "messaging",
    "region": "ap-south-1",
    "capability_ref": "event_streaming_layer",
    "data": {
      "label": "Event Streaming",
      "description": "Message broker"
    }
  },
  {
    "id": "db_primary",
    "type": "data",
    "category": "data",
    "region": "ap-south-1",
    "capability_ref": "managed_nosql_db",
    "data": {
      "label": "Primary Database",
      "description": "Managed NoSQL"
    },
    "config": {
      "public_access": false,
      "multi_az": true
    }
  },
  {
    "id": "db_replica",
    "type": "data",
    "category": "data",
    "region": "ap-southeast-1",
    "capability_ref": "managed_nosql_db",
    "data": {
      "label": "Replica Database",
      "description": "Cross-region replica"
    },
    "config": {
      "public_access": false,
      "replication_role": "replica"
    }
  },
  {
    "id": "cache_1",
    "type": "cache",
    "category": "cache",
```

```
    "region": "ap-south-1",
    "capability_ref": "distributed_cache",
    "data": {
      "label": "Distributed Cache",
      "description": "In-memory cache"
    }
  }
],
"edges": [
  {
    "id": "e1",
    "from": "client",
    "to": "waf_1",
    "protocol": "https",
    "type": "traffic"
  },
  {
    "id": "e2",
    "from": "waf_1",
    "to": "api_gw_1",
    "protocol": "https",
    "type": "traffic"
  },
  {
    "id": "e3",
    "from": "api_gw_1",
    "to": "compute_1",
    "protocol": "http",
    "type": "traffic"
  },
  {
    "id": "e4",
    "from": "compute_1",
    "to": "event_stream_1",
    "protocol": "tcp",
    "type": "data"
  },
  {
    "id": "e5",
    "from": "compute_1",
    "to": "db_primary",
    "protocol": "tcp",
    "type": "data"
  },
  {
    "id": "e6",
    "from": "compute_1",
    "to": "cache_1",
    "protocol": "tcp",
```

```
        "type": "data"
      },
      {
        "id": "e7",
        "from": "db_primary",
        "to": "db_replica",
        "protocol": "replication",
        "type": "replication",
        "replication_mode": "active_active"
      }
    ]
  }
}
```

15.7 Resolved Architecture JSON (Graph + Product Mapping per Node)


```
{
  "architecture_id": "arch_001",
  "graph": {
    "nodes": [
      {
        "id": "waf_1",
        "type": "security",
        "capability_ref": "waf_layer",
        "data": { "label": "WAF" },
        "resolved_products": [
          {
            "provider": "your_cloud",
            "product_name": "YC WAF",
            "product_id": "yc-waf",
            "recommended": true
          },
          {
            "provider": "aws",
            "product_name": "AWS WAF",
            "product_id": "aws-waf",
            "recommended": false
          }
        ]
      },
      {
        "id": "compute_1",
        "type": "compute",
        "capability_ref": "microservices_compute",
        "data": { "label": "Application Servers" },
        "resolved_products": [
          {
            "provider": "your_cloud",
            "product_name": "YC Standard VM",
            "product_id": "yc-vm-standard",
            "recommended": true
          },
          {
            "provider": "aws",
            "product_name": "Amazon EC2",
            "product_id": "aws-ec2",
            "recommended": false
          }
        ]
      },
      {
        "id": "db_primary",
```

```
        "type": "data",
        "capability_ref": "managed_nosql_db",
        "data": { "label": "Primary Database" },
        "resolved_products": [
          {
            "provider": "your_cloud",
            "product_name": "YC Managed NoSQL",
            "product_id": "yc-managed-nosql",
            "recommended": true
          },
          {
            "provider": "aws",
            "product_name": "Amazon DynamoDB",
            "product_id": "aws-dynamodb",
            "recommended": false
          }
        ]
      }
    ],
    "edges": []
  },
  "estimated_monthly_cost_usd": 18500
}
```

15.8 Compliance Report JSON (All Scenarios)


```
{
  "compliance_status": "compliant",
  "risk_score": 0.02,
  "sector": "BFSI",
  "policy_id": "bfsi_baseline_v1",
  "evaluated_at": "2026-02-26T10:05:00Z",
  "checks": [
    {
      "rule_id": "data_encryption_at_rest_required",
      "status": "pass",
      "target_category": "data",
      "details": "All data nodes support encryption at rest"
    },
    {
      "rule_id": "data_multi_region_required",
      "status": "pass",
      "target_category": "data",
      "details": "Data deployed in 2 regions with replication"
    },
    {
      "rule_id": "no_public_database_access",
      "status": "pass",
      "target_category": "data",
      "details": "No public_access on data nodes"
    },
    {
      "rule_id": "waf_required_for_public_api",
      "status": "pass",
      "target_category": "api_gateway",
      "details": "WAF precedes API gateway in graph"
    },
    {
      "rule_id": "latency_threshold",
      "status": "pass",
      "target_category": "compute",
      "details": "P95 latency within 300ms"
    },
    {
      "rule_id": "data_residency_constraint",
      "status": "pass",
      "target_category": "data",
      "details": "Regions within ap-south-1, ap-southeast-1"
    }
  ],
  "violations": [],
  "auto_remediations_applied": [
```

```
    "added_key_management_service",
    "added_centralized_logging",
    "added_secondary_data_region"
  ],
  "forbidden_patterns_checked": [
    {
      "pattern_id": "single_point_of_failure_db",
      "status": "pass"
    },
    {
      "pattern_id": "public_database",
      "status": "pass"
    }
  ]
}
```

Example with violations (for reference):

```
{
  "compliance_status": "non_compliant",
  "risk_score": 0.35,
  "violations": [
    {
      "rule_id": "data_multi_region_required",
      "severity": "critical",
      "message": "Data nodes found in only 1 region",
      "affected_nodes": ["db_primary"]
    }
  ],
  "auto_remediations_available": [
    {
      "action": "add_replication_region",
      "parameters": { "min_regions": 2 }
    }
  ]
}
```

15.9 Execution Plan JSON (Deploy DAG)


```
{
  "execution_plan_id": "ep_001",
  "architecture_id": "arch_001",
  "created_at": "2026-02-26T10:10:00Z",
  "steps": [
    {
      "step_id": "step_1",
      "order": 1,
      "action": "create_network",
      "target_block": "vpc",
      "provider": "your_cloud",
      "dependencies": []
    },
    {
      "step_id": "step_2",
      "order": 2,
      "action": "create_security_groups",
      "target_block": "security",
      "provider": "your_cloud",
      "dependencies": ["step_1"]
    },
    {
      "step_id": "step_3",
      "order": 3,
      "action": "deploy_waf",
      "target_block": "waf_layer",
      "node_id": "waf_1",
      "provider": "your_cloud",
      "dependencies": ["step_2"]
    },
    {
      "step_id": "step_4",
      "order": 4,
      "action": "deploy_api_gateway",
      "target_block": "api_gateway",
      "node_id": "api_gw_1",
      "provider": "your_cloud",
      "dependencies": ["step_3"]
    },
    {
      "step_id": "step_5",
      "order": 5,
      "action": "deploy_compute",
      "target_block": "microservices_compute",
      "node_id": "compute_1",
      "provider": "your_cloud",
```

```
      "dependencies": ["step_4"]
    },
    {
      "step_id": "step_6",
      "order": 6,
      "action": "deploy_database",
      "target_block": "managed_nosql_db",
      "node_id": "db_primary",
      "provider": "your_cloud",
      "dependencies": ["step_2"]
    },
    {
      "step_id": "step_7",
      "order": 7,
      "action": "deploy_database_replica",
      "target_block": "managed_nosql_db",
      "node_id": "db_replica",
      "provider": "your_cloud",
      "dependencies": ["step_6"]
    },
    {
      "step_id": "step_8",
      "order": 8,
      "action": "configure_replication",
      "target_block": null,
      "dependencies": ["step_6", "step_7"]
    },
    {
      "step_id": "step_9",
      "order": 9,
      "action": "deploy_cache",
      "target_block": "distributed_cache",
      "node_id": "cache_1",
      "provider": "aws",
      "dependencies": ["step_5"]
    }
  ]
}
```

15.10 Deployment Request JSON (Per Provider)

Your cloud:

```
{
  "deployment_request_id": "dr_001",
  "execution_plan_id": "ep_001",
  "step_id": "step_5",
  "provider": "your_cloud",
  "action": "deploy_compute",
  "payload": {
    "product_id": "yc-vm-standard",
    "region": "ap-south-1",
    "node_id": "compute_1",
    "config": {
      "scale_type": "horizontal",
      "min_instances": 2,
      "max_instances": 10,
      "network_id": "vpc-xxx",
      "security_group_ids": ["sg-xxx"]
    }
  }
}
```

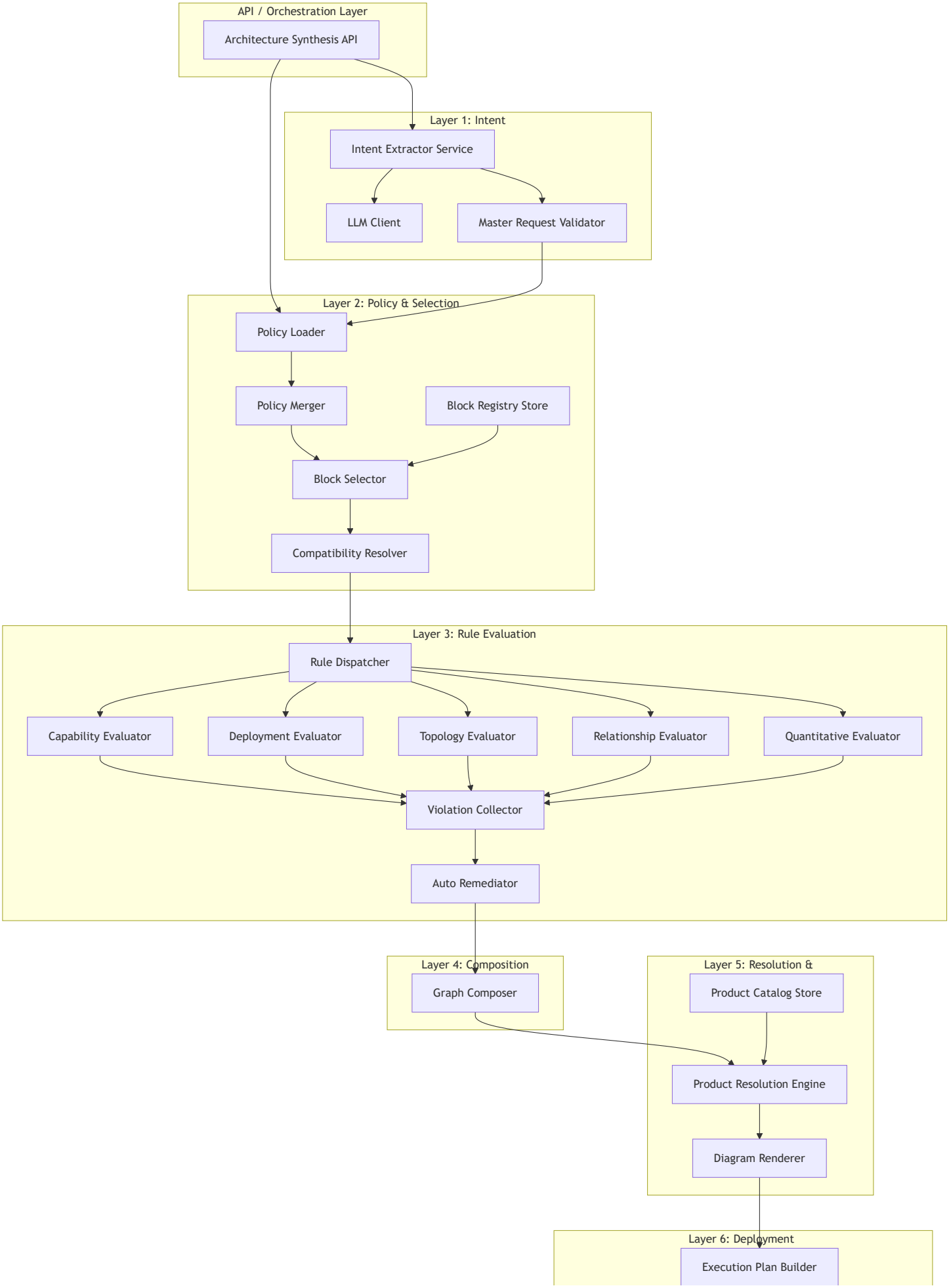

AWS (Terraform):

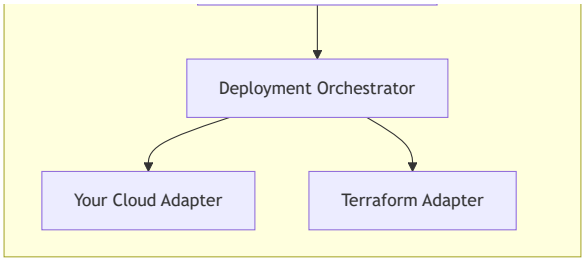

```
{
  "deployment_request_id": "dr_002",
  "execution_plan_id": "ep_001",
  "step_id": "step_9",
  "provider": "terraform_aws",
  "action": "deploy_cache",
  "payload": {
    "terraform_module": "aws_elasticache_redis",
    "variables": {
      "cluster_id": "cache-arch-001",
      "node_type": "cache.t3.micro",
      "num_cache_nodes": 2,
      "region": "ap-south-1"
    }
  }
}
```


16. Low Level Design (LLD)

This section describes the component-level design, interfaces, sequences, and technical details for implementing the Architecture Synthesis Platform.

16.1 Component Diagram


API / Orchestration Layer
Architecture Synthesis API
Layer 1: Intent
Intent Extractor Service
LLM Client
Master Request Validator
Layer 2: Policy & Selection
Policy Loader
Policy Merger
Block Registry Store
Block Selector
Compatibility Resolver
Layer 3: Rule Evaluation
Rule Dispatcher
Capability Evaluator
Deployment Evaluator
Topology Evaluator
Relationship Evaluator
Quantitative Evaluator
Violation Collector
Auto Remediator
Layer 4: Composition
Graph Composer
Layer 5: Resolution &
Product Catalog Store
Product Resolution Engine
Diagram Renderer
Layer 6: Deployment
Execution Plan Builder



16.2 Module Breakdown

Module	Responsibility	Inputs	Outputs
Intent Extractor Service	Call LLM, parse response, validate against Master Request schema	Chat history, products summary, personalized_data	Master Request JSON
Master Request Validator	Schema validation, normalization, defaulting	Raw LLM output	Validated Master Request JSON
Policy Loader	Load policy files by sector/region/org from store (DB/Redis/files)	sector, region, org_level	List of Policy JSONs
Policy Merger	Merge multiple policies, resolve conflicts by priority/strategy	List of Policy JSONs	Merged policy (rules + metadata)
Block Registry Store	Read-only access to block definitions	—	Block definitions by id/category
Block Selector	Map intent + merged policy to list of block_ids (scoring, mandatory_blocks)	Master Request, merged policy	selected_blocks, deployment_topology hints
Compatibility Resolver	Resolve requires, add missing blocks, remove conflicts	selected_blocks, Block Registry	Validated block set
Rule Dispatcher	Route each rule to correct evaluator by rule_type	Merged policy rules, graph (or block set)	Per-rule evaluation results
Capability Evaluator	Check node/block capabilities against condition	rule, node or block metadata	pass/fail, details
Deployment Evaluator	Check region_count, replication, data_residency	rule, graph nodes / deployment_topology	pass/fail, details
Topology Evaluator	Check public_access, layer placement	rule, graph	pass/fail, details

Module	Responsibility	Inputs	Outputs
Relationship Evaluator	Check must_be_preceded_by, edge paths	rule, graph edges	pass/fail, details
Quantitative Evaluator	Check latency, cost thresholds	rule, aggregated metrics	pass/fail, details
Violation Collector	Aggregate results, compute risk_score	Evaluator outputs	violations list, risk_score
Auto Remediator	Apply add_block, add_replication_region, insert_block_before	violations with auto_remediation, graph/block set	Modified graph/block set
Graph Composer	Build nodes and edges from block set + topology rules	Validated blocks, deployment_topology	Graph Topology JSON
Product Catalog Store	Read-only access to product catalogs per provider	—	Product list by provider/capability
Product Resolution Engine	Map each graph node (capability_ref) to products per provider	Graph, Product Catalogs, preferences	Resolved Architecture JSON
Diagram Renderer	Convert graph + resolved products to frontend format (e.g. Cytoscape/React Flow)	Resolved Architecture	Diagram payload + hover metadata
Execution Plan Builder	Topological sort of nodes, build ordered steps with dependencies	Graph Topology, Resolved Architecture	Execution Plan JSON
Deployment Orchestrator	Execute steps in order, call adapters, track state	Execution Plan	Deployment status
Your Cloud Adapter	Translate step payload to your cloud API calls	Deployment Request (your_cloud)	API response
Terraform Adapter	Generate/apply Terraform for AWS/Azure/GCP	Deployment Request (terraform_*)	Terraform state / output

16.3 Interface Contracts (Key Services)

16.3.1 Intent Extractor Service

```
Interface: IIntentExtractor

  generate_master_request(
    chat_history: List[ChatMessage],
    products_summary: Optional[Dict],
    personalized_data: Optional[Dict],
    sector: str = "BFSI"
  ) -> Async[MasterRequestJSON]

  - Calls LLM with structured prompt and schema description.
  - Parses JSON from LLM response.
  - Delegates to Master Request Validator.
  - Raises: IntentExtractionError on parse/validation failure.
```

16.3.2 Policy Loader & Merger

```
Interface: IPolicyLoader

  load_policies(
    sector: str,
    region: Optional[str] = None,
    org_level: Optional[str] = None
  ) -> Async[List[PolicyJSON]]

  - Loads from: policies/sectors/{sector}.json, policies/regional/{region}.json, policies/org/{org_level}.js
  - Returns list of policy objects (same unified schema).

Interface: IPolicyMerger

  merge(
    policies: List[PolicyJSON],
    strategy: ConflictResolutionStrategy = "highest_priority_wins"
  ) -> MergedPolicy

  - Merges rules arrays; deduplicates by rule_id (priority wins).
  - Merges mandatory_blocks, forbidden_patterns.
  - Returns single MergedPolicy (same schema, combined rules).
```

16.3.3 Block Selector

```
Interface: IBlockSelector

  select_blocks(
    master_request: MasterRequestJSON,
    merged_policy: MergedPolicy,
    block_registry: BlockRegistry
  ) -> SelectedArchitectureJSON

  - Translates intent to required capabilities.
  - Adds mandatory_blocks from policy.
  - Scores blocks from registry against capabilities/optimization_priorities.
  - Returns selected_blocks, deployment_topology, scoring_summary, estimated_monthly_cost_usd.
```

16.3.4 Compatibility Resolver

```
Interface: ICompatibilityResolver

  resolve(
    selected_block_ids: List[str],
    block_registry: BlockRegistry
  ) -> Tuple[List[BlockDefinition], List[CompatibilityWarning]]

  - For each block, resolves requires (adds missing blocks).
  - Removes or replaces blocks that conflict.
  - Returns ordered list of block definitions and any warnings.
```

16.3.5 Rule Evaluation Engine (Dispatcher + Evaluators)

```
Interface: IRuleEvaluator (per rule_type)

  evaluate(
    rule: RuleDefinition,
    context: EvaluationContext  # graph or block set + deployment_topology
  ) -> RuleEvaluationResult

  - CapabilityEvaluator: checks context node/block capabilities.
  - DeploymentEvaluator: checks region_count, replication, residency.
  - TopologyEvaluator: checks graph structure (e.g. public_access).
  - RelationshipEvaluator: graph traversal (e.g. WAF before API).
  - QuantitativeEvaluator: aggregates metrics, compares to threshold.

Interface: IRuleEvaluationEngine

  evaluate_all(
    merged_policy: MergedPolicy,
    graph: GraphTopologyJSON
  ) -> ComplianceReportJSON

  - Dispatches each rule to correct evaluator.
  - Collects results into ViolationCollector.
  - If auto_remediation enabled, calls AutoRemediator and re-evaluates.
  - Returns ComplianceReportJSON.
```

16.3.6 Graph Composer

```
Interface: IGraphComposer

  compose(
    blocks: List[BlockDefinition],
    deployment_topology: DeploymentTopology,
    topology_constraints: Optional[List[TopologyConstraint]] = None
  ) -> GraphTopologyJSON

  - Creates nodes from blocks (id, type, category, region, capability_ref, config).
  - Creates edges from block interfaces (ingress/egress) and layering rules.
  - Adds replication edges when deployment_topology.replication_required.
  - Applies topology_constraints (e.g. no public DB).
  - Returns graph with nodes and edges.
```

16.3.7 Product Resolution Engine

```
Interface: IProductResolutionEngine

  resolve(
    graph: GraphTopologyJSON,
    product_catalogs: Dict[str, ProductCatalogJSON],
    preferences: Optional[DeploymentPreferences] = None
  ) -> ResolvedArchitectureJSON

  - For each node with capability_ref, finds matching products from each catalog.
  - Applies preferences (e.g. prefer your_cloud) to set recommended flag.
  - Attaches resolved_products to each node; returns ResolvedArchitectureJSON.
```

16.3.8 Execution Plan Builder

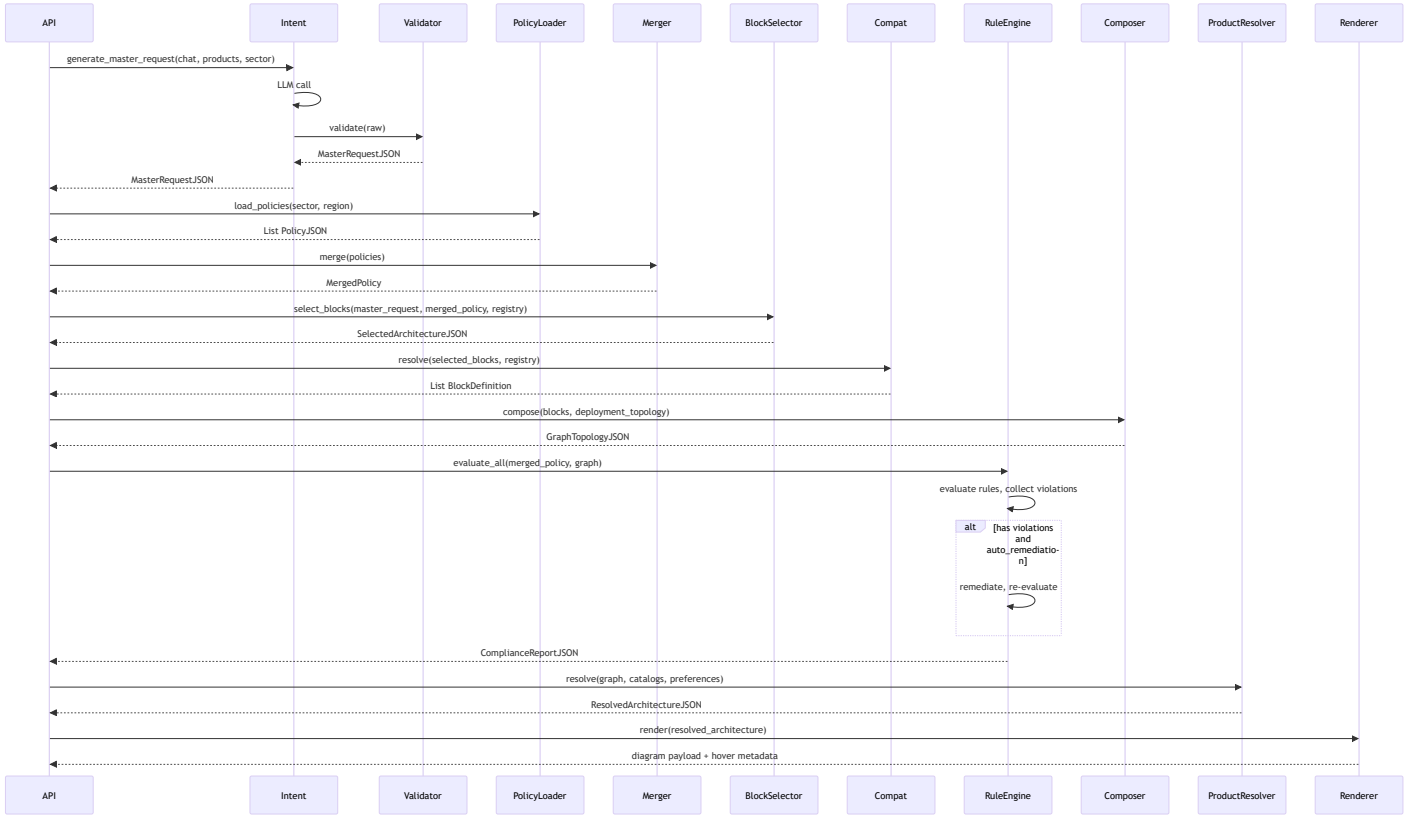
```
Interface: IExecutionPlanBuilder

  build(
    graph: GraphTopologyJSON,
    resolved_architecture: ResolvedArchitectureJSON
  ) -> ExecutionPlanJSON

  - Topological sort of nodes (network → security → compute → data → replication).
  - One step per deployable node/group; dependencies from graph edges.
  - Each step: step_id, order, action, target_block, node_id, provider, dependencies.
```

16.3.9 Deployment Orchestrator & Adapters

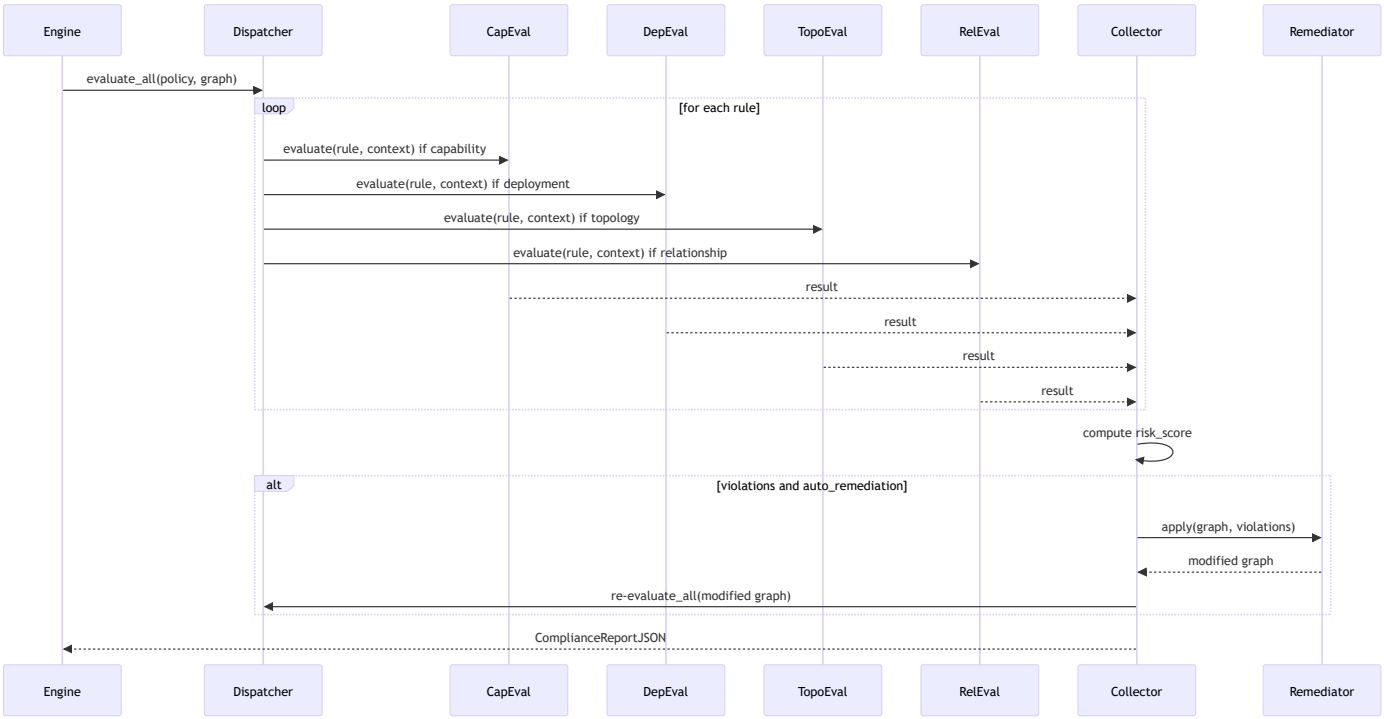
```
Interface: IProviderAdapter

  deploy(
    deployment_request: DeploymentRequestJSON
  ) -> Async[DeploymentResult]

  - YourCloudAdapter: maps to your cloud API (e.g. create_vm, create_db).
  - TerraformAdapter: writes Terraform vars, runs terraform apply (or delegates to job).

Interface: IDeploymentOrchestrator

  execute(
    execution_plan: ExecutionPlanJSON,
    adapters: Dict[str, IProviderAdapter]
  ) -> Async[DeploymentStatus]

  - Executes steps in order; waits for dependencies.
  - Builds DeploymentRequestJSON per step; invokes correct adapter by provider.
  - Tracks state (pending, in_progress, failed, completed); supports rollback on failure.
```

16.4 Key Sequence Diagrams

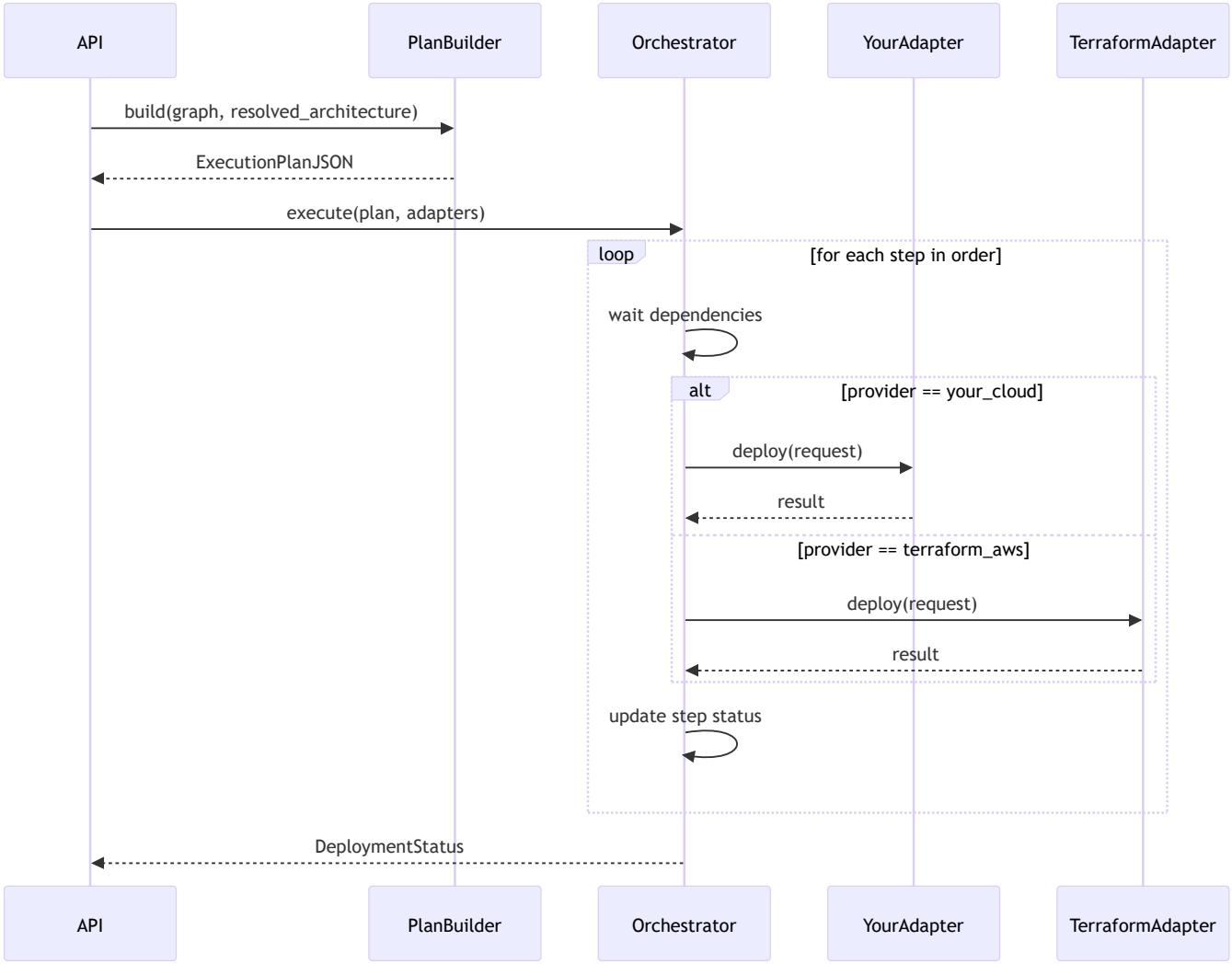
16.4.1 End-to-End: Questionnaire to Diagram



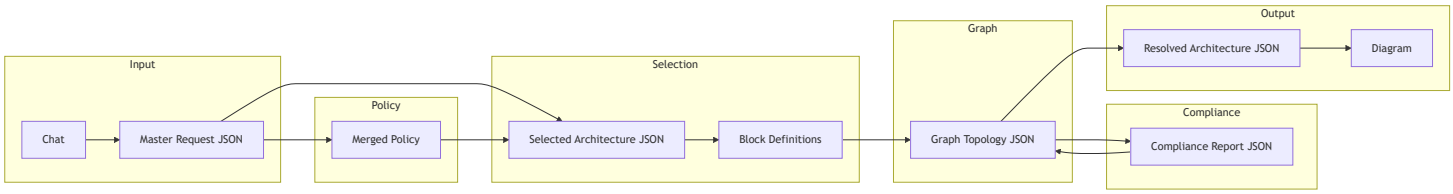
16.4.2 Compliance Evaluation (Rule Dispatcher → Evaluators)



16.4.3 Deployment Execution



16.5 Data Flow Between Layers



16.6 Configuration & Extensibility

Concern	Recommendation
Policy store	DB table (sector, region, org, policy_json) + optional Redis cache; or file-based under `/policies` with same schema.

Concern	Recommendation
Block registry	JSON files or DB; load at startup or on-demand; versioned by block_id + version.
Product catalogs	Per-provider JSON or API; refresh periodically; support override per tenant.
LLM	Config: model_id, temperature, max_tokens; prompt templates in files or DB.
Rule evaluators	Plugin registry by rule_type; new rule types = new evaluator class implementing IRuleEvaluator.
Provider adapters	Registry by provider key (your_cloud, terraform_aws, terraform_azure); new cloud = new adapter implementing IProviderAdapter.
Logging	Structured logs (request_id, session_id, layer, duration); log rule violations and remediation actions for audit.
Errors	Domain exceptions: IntentExtractionError, PolicyNotFoundError, CompatibilityError, ComplianceViolationError, DeploymentStepFailedError; map to HTTP and user messages.

16.7 Error Handling Strategy

Layer	Error Types	Handling
Intent	LLM timeout, invalid JSON, schema validation failure	Retry LLM once; return 400 with message "Unable to interpret requirements"; log raw response.
Policy	Missing sector/region policy, invalid rule schema	Return 503 "Policy not available"; log missing policy key.
Block selection	No blocks match intent	Return 400 "No suitable architecture found for constraints"; suggest relaxing constraints.
Compatibility	Irresolvable conflicts	Return 400 with list of conflicting blocks; suggest alternatives.
Compliance	Non-compliant and strict mode	Return 422 with ComplianceReport (violations); do not return graph until compliant or user overrides.
Graph composer	Topology constraint violation	Auto-remediate if possible; else return 422 with violation details.
Product resolution	No product for capability in preferred provider	Fallback to next provider; log; if none, return 503 "No product mapping for capability X".

Layer	Error Types	Handling
Deployment	Step failure (API or Terraform error)	Mark step failed; optional rollback completed steps; return 500 with step_id and error; persist state for retry.

16.8 Suggested Package/Class Layout (Python)

```
app/
  architecture_synthesis/
    api.py                    # FastAPI router: /synthesize, /deploy
    intent/
      intent_extractor.py     # IntentExtractorService, IIntentExtractor
      master_request_validator.py
      schemas.py              # MasterRequestJSON Pydantic model
    policy/
      policy_loader.py        # PolicyLoader (file or DB)
      policy_merger.py        # PolicyMerger
      policy_schemas.py       # PolicyJSON, Rule, etc.
    blocks/
      block_registry.py       # BlockRegistryStore
      block_selector.py       # BlockSelector
      compatibility_resolver.py
      block_schemas.py
    compliance/
      rule_engine.py          # RuleEvaluationEngine, RuleDispatcher
      evaluators/
        capability_evaluator.py
        deployment_evaluator.py
        topology_evaluator.py
        relationship_evaluator.py
        quantitative_evaluator.py
      violation_collector.py
      auto_remediator.py
      compliance_schemas.py  # ComplianceReportJSON
    graph/
      graph_composer.py       # GraphComposer
      graph_schemas.py       # GraphTopologyJSON (nodes, edges)
    resolution/
      product_catalog.py      # ProductCatalogStore
      product_resolver.py     # ProductResolutionEngine
      resolved_schemas.py
    diagram/
      diagram_renderer.py     # Format for frontend
    deployment/
      execution_plan_builder.py
      deployment_orchestrator.py
      adapters/
        base.py              # IProviderAdapter
        your_cloud_adapter.py
        terraform_adapter.py
      deployment_schemas.py
```

16.9 Summary

The LLD adds:

- **Component diagram**: All layers from API through Intent, Policy, Block Selection, Rule Evaluation, Graph Composer, Product Resolution, Diagram Renderer, and Deployment.
- **Module breakdown**: Responsibility, inputs, and outputs for each module.
- **Interface contracts**: Method-level behavior for Intent Extractor, Policy Loader/Merger, Block Selector, Compatibility Resolver, Rule Evaluation Engine, Graph Composer, Product Resolution Engine, Execution Plan Builder, and Deployment Orchestrator/Adapters.
- **Sequence diagrams**: End-to-end (questionnaire to diagram), compliance evaluation with dispatcher and remediator, and deployment execution.
- **Data flow**: How Master Request, Policy, Selected Architecture, Graph, Compliance Report, and Resolved Architecture move between layers.
- **Configuration and extensibility**: Policy/block/product storage, LLM config, rule and provider plugins, logging.
- **Error handling**: Per-layer error types and handling strategy.
- **Package layout**: Suggested Python module structure for implementation.

This LLD is intended to be implementation-ready for the Architecture Synthesis Platform.

17. Complete Filter Logic: Intent → Architecture Diagram

This section documents the full filter logic: which Master Request (Intent) fields map to what, and how we generate the architecture diagram from intent.

17.1 Overview: Intent → Diagram Flow

```
Master Request (Intent)
        ↓
Policy loading (by sector/region/org)
        ↓
Categories needed
        ↓
Required capabilities per category
        ↓
Block selection (filter + score)
        ↓
Deployment topology (regions, replication, counts)
        ↓
Compatibility resolution (requires, conflicts)
        ↓
Graph composition (nodes + edges)
        ↓
Compliance evaluation
        ↓
Product resolution
        ↓
Architecture diagram (nodes with products)
```

17.2 Field-by-Field Mapping: Intent → Architecture

17.2.1 Request Metadata

Intent Field	Maps To	Effect
request_metadata.sector	Policy loading	Load policy for that sector (e.g. BFSI).
request_metadata.region	Policy loading	Load regional policy (e.g. RBI India).
request_metadata.environment	Policy applicability	Apply production vs non-production rules.
request_metadata.business_criticality	Block selection, scoring	Prefer HA/compliance blocks; higher min instances for HA.

17.2.2 Functional Requirements

Intent Field	Maps To	Effect
functional_requirements.application_type	Categories, capabilities	`web_application` → need network (api_gateway), security (waf),

Intent Field	Maps To	Effect
		compute, data. `batch_job` → compute, storage, maybe messaging.
functional_requirements.interaction_model	Categories, capabilities	`synchronous` → may skip messaging. `asynchronous` / `event_driven` → need messaging block with `asynchronous_processing`, `event_driven_architecture`.
functional_requirements.real_time_processing	Category, capabilities	Need **messaging** block; require `asynchronous_processing` or `high_throughput_streaming`.
functional_requirements.batch_processing	Category, capabilities	Need **storage** or **data** for batch; prefer blocks with batch-related capabilities.
functional_requirements.public_access_required	Categories, topology	Need **security** (waf), **network** (load balancer, api_gateway). External → WAF → API GW flow.
functional_requirements.api_required	Category	Need **network** block (api_gateway) with `api_management`, `routing`.
functional_requirements.third_party_integrations	Categories, capabilities	Need **network** (api_gateway), **security**; require `api_management`, `routing`.

17.2.3 Non-Functional Requirements

Intent Field	Maps To	Effect
non_functional_requirements.availability_target_percent	Counts, capabilities	99.9+ → require `multi_az_deployment`, min 2 instances for compute/data (HA).
non_functional_requirements.latency_p95_ms	Block selection	Prefer blocks with `latency_p95_ms ≤ value`.
non_functional_requirements.expected_rps_peak	Counts, block selection	Higher RPS → more compute instances (or min_instances); prefer blocks with higher `max_rps_supported`.
non_functional_requirements.daily_active_users	Counts	Higher DAU → more instances or higher scale;

Intent Field	Maps To	Effect
		affects min_instances for compute.
non_functional_requirements.horizontal_scaling_required	Capabilities, counts	Require `horizontal_scaling`, `stateless_compute` for compute; min 2 instances for HA.
non_functional_requirements.multi_region_required	Deployment topology, counts	`regions: [region1, region2]`; data block → 2 nodes (primary + replica); compute → 1 per region or 2+ in primary.
non_functional_requirements.disaster_recovery (RTO/RPO)	Deployment topology, capabilities	Replication; require data blocks with `multi_az_deployment`; 2 data nodes (primary + replica).

17.2.4 Data Requirements

Intent Field	Maps To	Effect
data_requirements.data_types	Category, capabilities	`structured` → **data** block with `structured_storage`. `unstructured` → **storage** block with `unstructured_storage`. Both → both blocks.
data_requirements.consistency_model	Capabilities	`eventual` → require `eventual_consistency`. `strong` → prefer blocks with strong consistency.
data_requirements.retention_years	Capabilities	Prefer blocks with retention/archive support.
data_requirements.analytics_required	Category, capabilities	Add **storage** or **data** block suitable for analytics.
data_requirements.real_time_analytics	Category, capabilities	May need **messaging** or streaming block for real-time analytics.

17.2.5 Security Requirements

Intent Field	Maps To	Effect
security_requirements.authentication_required	Topology	Ensure auth layer in graph (e.g. API GW or auth service).

Intent Field	Maps To	Effect
security_requirements.authorization_model	Capabilities	`rbac` → prefer blocks with RBAC-related capabilities.
security_requirements.data_encryption_at_rest	Capabilities	Require `data_encryption_at_rest: true` for **data**, **storage** blocks.
security_requirements.data_encryption_in_transit	Capabilities	Require `data_encryption_in_transit: true` for **data**, **storage**, **network** blocks.
security_requirements.audit_logging_required	Capabilities, categories	Require `audit_logging: true` where policy applies; add **observability** block (centralized_logging) if policy mandates.

17.2.6 Deployment Preferences

Intent Field	Maps To	Effect
deployment_preferences.cloud_provider_preference	Product resolution	Used later for preferred provider; can influence block `compliance_tags` or region matching.
deployment_preferences.multi_region_required	Deployment topology	Same as non_functional; set `regions` , replication for data.
deployment_preferences.containerization_preferred	Capabilities	Require `container_support: true` for **compute**; filter compute blocks by this.
deployment_preferences.serverless_preferred	Block selection	Prefer serverless-style compute block if present in registry.
deployment_preferences.infrastructure_as_code_required	(No direct block impact)	Used for deployment (Terraform/IaC).

17.2.7 Optimization Priorities

Intent Field	Maps To	Effect
optimization_priorities.performance_weight	Block scoring	Higher weight → prefer blocks with higher `max_rps_supported`, lower `latency_p95_ms`.
optimization_priorities.cost_weight	Block scoring	Higher weight → prefer blocks with lower `base_monthly_cost_usd`.
optimization_priorities.compliance_weight	Block scoring	Higher weight → prefer blocks with more `compliance_tags` (e.g. BFSI, PCI-DSS).
optimization_priorities.operational_simplicity_weight	Block scoring	Higher weight → prefer blocks with lower `complexity_score`.

17.2.8 Budget Constraints

Intent Field	Maps To	Effect
budget_constraints.monthly_budget_usd	Block selection	Reject or down-rank blocks that push total cost over budget.
budget_constraints.cost_optimization_priority	Block scoring	Higher priority → stronger cost weight in scoring.

17.3 Categories Required by Intent

Intent Signals	Categories Needed
Web app, API, public access, third-party	**network** (api_gateway), **security** (waf), **compute**
Persistent data	**data** (and/or **storage** if unstructured)
Real-time, async, event-driven	**messaging**
Multi-region, HA	**data** (with replication), **compute** (multi_az / redundancy)
Audit, encryption, compliance	**security** (key_management), **observability** (centralized_logging)
Load balancing	**network** (load_balancer)

17.4 Required Capabilities per Category

Category	Capabilities Typically Required (from Intent + Policy)
compute	`stateless_compute`, `horizontal_scaling`, `container_support` (if containerization_preferred)
data	`data_encryption_at_rest`, `data_encryption_in_transit`, `audit_logging`, `multi_az_deployment` (if HA), `structured_storage` or `eventual_consistency` (from data_types, consistency_model)
storage	`data_encryption_at_rest`, `audit_logging`, `unstructured_storage` (if unstructured data)
security	`web_application_firewall`, `audit_logging`, `ddos_protection`
network	`api_management`, `routing`, `rate_limiting`
messaging	`asynchronous_processing`, `event_driven_architecture`, `high_throughput_streaming`
cache	`in_memory_cache`, `horizontal_scaling`, `low_latency`
observability	`audit_logging`, `log_aggregation`, `retention`

17.5 Counts / Instances from Intent

Intent Field	Effect on Counts
`multi_region_required: true`	Data: 2 nodes (primary + replica). Compute: 1 per region or 2+ in primary.
`availability_target_percent` 99.9+	Min 2 instances for compute and data (HA).
`horizontal_scaling_required: true`	Compute: min 2 instances; max from RPS/users or defaults.
`disaster_recovery` (RTO/RPO)	Replication → 2 data nodes (or more).
Policy: "2+ backends for LB"	Min 2 compute nodes.

17.6 End-to-End Filter Logic (Pseudo-Code)

```
// Step 1: Load policy
policy = load_policy(sector=request_metadata.sector, region=request_metadata.region)
add mandatory_blocks from policy

// Step 2: Derive categories needed
categories_needed = []
if application_type in [web_application, api] or public_access_required or api_required:
    categories_needed += [network, security, compute]
if data_types includes structured: categories_needed += [data]
if data_types includes unstructured: categories_needed += [storage]
if real_time_processing or interaction_model == asynchronous:
    categories_needed += [messaging]
if multi_region_required or availability_target_percent >= 99.9:
    ensure data, compute have redundancy
if audit_logging_required or policy mandates:
    categories_needed += [observability]
categories_needed += policy.mandatory_blocks (by block category)

// Step 3: Derive required capabilities per category
for category in categories_needed:
    required_caps[category] = []
    from security_requirements: add data_encryption_at_rest, data_encryption_in_transit, audit_logging
    from policy capability rules where target_category == category:
        add condition.field where condition.value == true
    from intent:
        if horizontal_scaling_required and category == compute: add stateless_compute, horizontal_scaling
        if containerization_preferred and category == compute: add container_support
        if real_time_processing and category == messaging: add asynchronous_processing
        if data_types.structured and category == data: add structured_storage
        if consistency_model == eventual and category == data: add eventual_consistency
        ... (per table in 17.2)

// Step 4: Filter and score blocks
for category in categories_needed:
    candidates = blocks where block.category == category
    candidates = candidates where for each cap in required_caps[category]:
        block.capabilities_provided[cap] == true
    candidates = resolve requires (add missing blocks), remove conflicts
    selected = score(candidates, optimization_priorities) and pick best
    selected_blocks += selected

// Step 5: Derive deployment topology and counts
deployment_topology.regions = [region1, region2] if multi_region_required else [default_region]
deployment_topology.active_active = true if multi_region_required
for block in selected_blocks:
    if block.category == data and (multi_region_required or disaster_recovery):
```

```
        count = 2  // primary + replica
    elif block.category == compute and (horizontal_scaling_required or availability_target_percent >= 99.9):
        min_instances = 2
    else:
        count = 1
    topology.block_counts[block.block_id] = count

// Step 6: Graph Composer
create nodes: for each block, create count nodes (or 1 node with config.min_instances)
assign regions from deployment_topology
create edges from block interfaces + layering rules
output Graph Topology JSON

// Step 7: Compliance evaluation
evaluate graph against merged policy; apply auto-remediation if needed

// Step 8: Product resolution
for each node with capability_ref:
    match block.capabilities_provided + block.category to Product Catalog
    attach resolved_products to node
output Resolved Architecture JSON

// Step 9: Diagram renderer
convert Resolved Architecture to frontend diagram payload
```

18. Resource Sizing: CPU, RAM, Disk Configuration

This section covers how to dynamically calculate resource configurations (CPU, RAM, disk) for cloud resources using formula-based approach.

18.1 Why Resource Sizing Is Needed

The plan so far defines:

- **Blocks** (types with capabilities)
- **Nodes** (instances of blocks)
- **Product mapping** (capability → product name)

But it doesn't specify **how many CPUs, how much RAM, disk size** each node should have.

Resource sizing fills this gap: it calculates `cpu_count`, `ram_gb`, `disk_gb` for each node based on **intent** (RPS, latency, data volume) and **block benchmarks**.

18.2 Where Resource Config Lives

Resource configuration goes in **node.config** in the Graph Topology JSON:

```
{
  "id": "compute_1",
  "capability_ref": "microservices_compute",
  "region": "ap-south-1",
  "config": {
    "cpu_count": 4,
    "ram_gb": 16,
    "disk_gb": 100,
    "disk_type": "ssd",
    "min_instances": 2,
    "max_instances": 10
  }
}
```

This is used by:

- **Product Resolution:** Match to product SKUs (e.g. "4 vCPU, 16 GB" → AWS t3.xlarge, YC standard-m).
- **Deployment:** Provision with those specs.
- **Compliance:** Quantitative rules (e.g. "min 16 GB RAM for data nodes").

18.3 Block Benchmarks

Add **resource_benchmarks** to each block in Block Registry:

```
{
  "block_id": "microservices_compute",
  "category": "compute",
  "capabilities_provided": { "stateless_compute": true, "horizontal_scaling": true },
  "resource_benchmarks": {
    "rps_per_cpu": 500,
    "ram_per_cpu_gb": 4,
    "min_cpu": 2,
    "min_ram_gb": 8,
    "disk_gb_default": 100
  }
}
```

```
{
  "block_id": "managed_nosql_db",
  "category": "data",
  "capabilities_provided": { "data_encryption_at_rest": true, "audit_logging": true },
  "resource_benchmarks": {
    "data_per_cpu_gb": 100,
    "ram_per_cpu_gb": 8,
    "min_cpu": 2,
    "min_ram_gb": 16,
    "disk_multiplier": 1.5
  }
}
```

What these mean:

- **`rps_per_cpu` :** How many requests per second one vCPU can handle (e.g. 500 for typical web app).
- **`ram_per_cpu_gb` :** How much RAM per vCPU (e.g. 4 GB per vCPU).
- **`min_cpu` , `min_ram_gb` :** Safety minimums (floor values, not restrictions).
- **`data_per_cpu_gb` :** How much data one vCPU can manage (for data blocks).
- **`disk_multiplier` :** Overhead factor for disk (e.g. 1.5x for indexes, temp space).

18.4 Formula-Based Resource Calculation

Compute blocks (e.g. microservices_compute)

```
def calculate_compute_resources(intent, benchmarks, policy=None):
    # CPU from RPS
    rps_per_cpu = benchmarks["rps_per_cpu"]
    expected_rps = intent["non_functional_requirements"]["expected_rps_peak"]
    cpu_count = max(benchmarks["min_cpu"], ceil(expected_rps / rps_per_cpu))

    # Adjust for latency (lower latency → more CPU)
    latency_target = intent["non_functional_requirements"]["latency_p95_ms"]
    if latency_target < 100:
        cpu_count = int(cpu_count * 1.5)

    # RAM from CPU
    ram_per_cpu = benchmarks["ram_per_cpu_gb"]
    ram_gb = max(benchmarks["min_ram_gb"], cpu_count * ram_per_cpu)

    # Disk (default for stateless compute)
    disk_gb = benchmarks["disk_gb_default"]

    # Apply policy constraints (e.g. "compute must have >= 16 GB RAM")
    if policy:
        ram_gb = apply_policy_constraints(ram_gb, "compute", policy)

    return {"cpu_count": cpu_count, "ram_gb": ram_gb, "disk_gb": disk_gb}
```

Data blocks (e.g. managed_nosql_db)

```python
def calculate_data_resources(intent, benchmarks, policy=None):
    # CPU from data volume
    data_per_cpu = benchmarks["data_per_cpu_gb"]
    initial_data = intent["data_requirements"]["initial_volume_gb"]
    cpu_count = max(benchmarks["min_cpu"], ceil(initial_data / data_per_cpu))

    # RAM from CPU
    ram_per_cpu = benchmarks["ram_per_cpu_gb"]
    ram_gb = max(benchmarks["min_ram_gb"], cpu_count * ram_per_cpu)

    # Disk from data volume + growth
    disk_multiplier = benchmarks["disk_multiplier"]
    monthly_growth = intent["data_requirements"]["monthly_growth_gb"]
    disk_gb = int(initial_data * disk_multiplier + monthly_growth * 12)

    # Apply policy constraints
    if policy:
        ram_gb = apply_policy_constraints(ram_gb, "data", policy)

    return {"cpu_count": cpu_count, "ram_gb": ram_gb, "disk_gb": disk_gb}
```

18.5 Handling Same Block, Different Sizes

Key principle: Resource sizing runs **per node**, not per block.

Example: 2 compute nodes, different load

Intent:

```json
{
  "non_functional_requirements": {
    "expected_rps_peak": 2000,
    "horizontal_scaling_required": true
  }
}
```

Block selected: `microservices_compute`

Graph Composer creates 2 nodes (for HA):

Option A: Split load explicitly

```
total_rps = 2000
num_nodes = 2

# Uneven split (e.g. 80/20 for primary/secondary)
node_1_rps = total_rps * 0.8  # 1600
node_2_rps = total_rps * 0.2  # 400

# Calculate per node
resources_1 = calculate_compute_resources(intent_with_rps(1600), benchmarks)
# → { cpu: 4, ram: 16, disk: 100 }

resources_2 = calculate_compute_resources(intent_with_rps(400), benchmarks)
# → { cpu: 2, ram: 8, disk: 100 }

# Create nodes
node_1 = { "id": "compute_1", "config": resources_1 }
node_2 = { "id": "compute_2", "config": resources_2 }
```

Option B: Even split (default)

```
total_rps = 2000
num_nodes = 2
rps_per_node = total_rps / num_nodes  # 1000

resources = calculate_compute_resources(intent_with_rps(1000), benchmarks)
# → { cpu: 2, ram: 8, disk: 100 }

# Both nodes get same size
node_1 = { "id": "compute_1", "config": resources }
node_2 = { "id": "compute_2", "config": resources }
```

Result: Same block, 2 nodes, different sizes (Option A) or same sizes (Option B).

18.6 Complete Example: Same Block Twice, Different Sizes

Scenario: Load balancer with 2 backend app servers; primary handles 80% of traffic.

Intent:

```
{
  "non_functional_requirements": {
    "expected_rps_peak": 2000,
    "latency_p95_ms": 200
  }
}
```

Block: `microservices_compute`

Benchmarks:

```
{
  "rps_per_cpu": 500,
  "ram_per_cpu_gb": 4,
  "min_cpu": 2,
  "min_ram_gb": 8,
  "disk_gb_default": 100
}
```

Resource sizing:

Node 1 (primary, 80% load):

```
rps = 2000 * 0.8 = 1600
cpu = max(2, ceil(1600 / 500)) = max(2, 4) = 4
ram = max(8, 4 * 4) = 16
disk = 100
```

Node 2 (secondary, 20% load):

```
rps = 2000 * 0.2 = 400
cpu = max(2, ceil(400 / 500)) = max(2, 1) = 2
ram = max(8, 2 * 4) = 8
disk = 100
```

Graph output:

```
{
  "nodes": [
    {
      "id": "compute_1",
      "capability_ref": "microservices_compute",
      "config": { "cpu_count": 4, "ram_gb": 16, "disk_gb": 100 }
    },
    {
      "id": "compute_2",
      "capability_ref": "microservices_compute",
      "config": { "cpu_count": 2, "ram_gb": 8, "disk_gb": 100 }
    }
  ]
}
```


Same block, two nodes, different sizes.

18.7 Pipeline Integration

Add **Resource Sizing Service** between Block Selector and Graph Composer:

Updated flow:

```
Block Selector
    ↓
Selected Architecture JSON (block_ids, deployment_topology)
    ↓
[Resource Sizing Service]  ← NEW
    ↓
block_resources = { block_id → { cpu, ram, disk } }
    ↓
Graph Composer (uses block_resources to set node.config)
    ↓
Graph Topology JSON (nodes with config: { cpu, ram, disk, ... })
```

18.8 Resource Sizing Service Interface

```python
class ResourceSizingService:
    """Calculate CPU, RAM, disk for blocks based on intent and benchmarks."""

    def __init__(self, block_registry: BlockRegistry):
        self.block_registry = block_registry

    def calculate_resources_for_blocks(
        self,
        selected_blocks: List[str],
        intent: MasterRequestJSON,
        deployment_topology: DeploymentTopology,
        policy: Optional[MergedPolicy] = None
    ) -> Dict[str, Dict[str, Any]]:
        """
        Calculate resources for all selected blocks.

        Returns:
            Dict mapping block_id → { cpu_count, ram_gb, disk_gb }
            (or block_id → List[resources] if multiple nodes per block)
        """
        block_resources = {}

        for block_id in selected_blocks:
            block = self.block_registry.get_block(block_id)
            benchmarks = block.get("resource_benchmarks", {})
            category = block["category"]

            # Calculate based on category
            if category == "compute":
                resources = self._calculate_compute(intent, benchmarks, policy)
            elif category == "data":
                resources = self._calculate_data(intent, benchmarks, policy)
            elif category == "cache":
                resources = self._calculate_cache(intent, benchmarks, policy)
            elif category == "storage":
                resources = self._calculate_storage(intent, benchmarks, policy)
            else:
                resources = self._default_resources(benchmarks)

            block_resources[block_id] = resources

        return block_resources

    def _calculate_compute(self, intent, benchmarks, policy):
        # See formula in 18.4
        pass
```

```
    def _calculate_data(self, intent, benchmarks, policy):
        # See formula in 18.4
        pass
```

18.9 Key Design Points

1. **Benchmarks are per block type** (not per node): One `microservices_compute` block has one `resource_benchmarks`.
2. **Formula runs per node**: If you create 2 nodes from the same block, formula can run twice with different inputs (e.g. different RPS split).
3. **`min_cpu`, `min_ram_gb` are floor values**: They prevent under-provisioning but don't restrict upward (you can have 2, 4, 8, 16+ vCPUs).
4. **Same block → different sizes**: Supported. Resource sizing runs per node; nodes from the same block can have different `config` (cpu, ram, disk).
5. **Policy can add constraints**: E.g. "data nodes must have ≥ 16 GB RAM" → formula applies `max(calculated_ram, policy_min_ram)`.

18.10 Complete Block Registry with Benchmarks


```
{
  "blocks": [
    {
      "block_id": "microservices_compute",
      "category": "compute",
      "capabilities_provided": {
        "stateless_compute": true,
        "horizontal_scaling": true,
        "container_support": true
      },
      "resource_benchmarks": {
        "rps_per_cpu": 500,
        "ram_per_cpu_gb": 4,
        "min_cpu": 2,
        "min_ram_gb": 8,
        "disk_gb_default": 100
      }
    },
    {
      "block_id": "managed_nosql_db",
      "category": "data",
      "capabilities_provided": {
        "structured_storage": true,
        "data_encryption_at_rest": true,
        "audit_logging": true
      },
      "resource_benchmarks": {
        "data_per_cpu_gb": 100,
        "ram_per_cpu_gb": 8,
        "min_cpu": 2,
        "min_ram_gb": 16,
        "disk_multiplier": 1.5
      }
    },
    {
      "block_id": "distributed_cache",
      "category": "cache",
      "capabilities_provided": {
        "in_memory_cache": true,
        "low_latency": true
      },
      "resource_benchmarks": {
        "cache_per_cpu_gb": 50,
        "ram_multiplier": 1.2,
        "min_cpu": 2,
        "min_ram_gb": 8,
```

```
        "disk_multiplier": 2
      }
    },
    {
      "block_id": "object_storage",
      "category": "storage",
      "capabilities_provided": {
        "unstructured_storage": true,
        "data_encryption_at_rest": true
      },
      "resource_benchmarks": {
        "min_cpu": 2,
        "min_ram_gb": 8,
        "disk_multiplier": 1.5
      }
    }
  ]
}
```

18.11 Example: Same Block Twice with Different Sizes

Scenario: Multi-region with different user distribution.

Intent:

```
{
  "non_functional_requirements": {
    "expected_rps_peak": 2000,
    "multi_region_required": true
  },
  "deployment_preferences": {
    "regions": ["ap-south-1", "ap-southeast-1"]
  }
}
```

Block selected: `microservices_compute`

Load distribution:

- Region A (ap-south-1): 70% of users → 1400 RPS
- Region B (ap-southeast-1): 30% of users → 600 RPS

Resource sizing:

Node 1 (Region A):

```
rps = 1400
cpu = max(2, ceil(1400 / 500)) = max(2, 3) = 3
ram = max(8, 3 * 4) = 12
disk = 100
```

Node 2 (Region B):

```
rps = 600
cpu = max(2, ceil(600 / 500)) = max(2, 2) = 2
ram = max(8, 2 * 4) = 8
disk = 100
```

Graph output:

```
{
  "nodes": [
    {
      "id": "compute_ap_south",
      "capability_ref": "microservices_compute",
      "region": "ap-south-1",
      "config": { "cpu_count": 3, "ram_gb": 12, "disk_gb": 100 }
    },
    {
      "id": "compute_ap_southeast",
      "capability_ref": "microservices_compute",
      "region": "ap-southeast-1",
      "config": { "cpu_count": 2, "ram_gb": 8, "disk_gb": 100 }
    }
  ]
}
```

Same block, two nodes, different sizes.

18.12 Summary

- **Resource sizing** calculates CPU/RAM/disk dynamically from intent using **formulas**.
- **Benchmarks** (per block type) define ratios (e.g. `rps_per_cpu`, `ram_per_cpu_gb`).
- **Formula** uses benchmarks + intent (RPS, latency, data volume) → resources.
- `min_cpu`, `min_ram_gb` are safety floors, not restrictions (you can have 2, 4, 8, 16+ vCPUs).
- **Same block can produce nodes with different sizes** (formula runs per node with different inputs).
- **Resource Sizing Service** sits between Block Selector and Graph Composer.
- **node.config** holds the calculated resources; used by Product Resolution and Deployment.

This plan and the Mermaid diagrams above form the complete map of the pipeline, all JSON types, low-level design, filter logic, and resource sizing for the architecture synthesis platform.